

Estrutura de Algoritmos para Programação Competitiva *

Izabel Chaves

1

Sumário

1	Análise de Complexidade	2
2	Grafos	2
2.1	BFS — $O(N + M)$	2
2.2	DFS — $O(N + M)$	3
2.3	Coloração por BFS — $O(N + M)$	3
2.4	Bellman-Ford — $O(NM)$	4
2.5	Floyd-Warshall — $O(N^3)$	5
2.6	Topological Sort — $O(N + M)$	5
2.7	SCC — Kosaraju — $O(V + E)$	6
2.8	LCA — $O(\log N)$	6
2.9	Fluxo de Dinic	7
2.10	Dijkstra — $O((N + M) \log N)$	9
2.11	Representação de Grafos	10
3	Estruturas de Dados	11
3.1	DSU (Disjoint Set Union)	11
3.2	Fenwick Tree	12
3.3	Soma — Segment Tree	13
4	Strings	14
4.1	KMP — $O(N + M)$	14
5	Matemática	15
5.1	Fórmulas Matemáticas	15

*<https://github.com/rainbowcrack/competitive-programming-material>

1. Análise de Complexidade

Algoritmo / Estrutura	Complexidade
BFS	$O(N + M)$
DFS	$O(N + M)$
0-1 BFS	$O(N + M)$
Dijkstra	$O((N + M) \log N)$
Bellman-Ford	$O(NM)$
Floyd-Warshall	$O(N^3)$
Topological Sort	$O(N + M)$
Kosaraju	$O(N + M)$
Kruskal	$O(M \log M)$
Dinic	$O(N^2 M)$
LCA	$O(\log N)$ por consulta
DSU	$O(\alpha(N))$ amortizado
Segment Tree	$O(\log N)$ por query/update
Fenwick Tree	$O(\log N)$ por query/update
KMP	$O(N)$
GCD	$O(\log(\min(a, b)))$
Fast Power	$O(\log B)$
Crivo de Eratóstenes	$O(N \log \log N)$
Binary Search	$O(\log N)$
Sort	$O(N \log N)$
Vector acesso	$O(1)$
Vector push_back	$O(1)$ amortizado
Vector insert/erase	$O(N)$
Set / Map	$O(\log N)$
Unordered Map / Set	$O(1)$ médio

Tabela 1. Complexidades mais comuns em Programação Competitiva

2. Grafos

2.1. BFS — $O(N + M)$

Algoritmo 1. Busca em Largura (BFS)

```
1 void bfs(int s) {  
2     queue<int> q;  
3  
4     vis[s] = 1;  
5     q.push(s);  
6  
7     while (!q.empty()) {
```

```

8      int u = q.front();
9      q.pop();
10
11     for (int v : adj[u]) {
12         if (!vis[v]) {
13             vis[v] = 1;
14             q.push(v);
15         }
16     }
17 }
18 }

```

Uso:

```

1 vector<vector<int>> adj(n);
2 vector<int> vis(n, 0);
3
4 bfs(0);

```

2.2. DFS — $O(N + M)$

Algoritmo 2. Busca em Profundidade (DFS)

```

1 void dfs(int u) {
2     vis[u] = 1;
3
4     for (int v : adj[u]) {
5         if (!vis[v])
6             dfs(v);
7     }
8 }

```

Uso:

```

1 vector<vector<int>> adj(n);
2 vector<int> vis(n, 0);
3
4 dfs(0);

```

2.3. Coloração por BFS — $O(N + M)$

Algoritmo 3. Coloração por BFS (verificação de bipartidos)

```

1 vector<int> color(n, -1);
2
3 bool bipartite = true;
4
5 for (int s = 0; s < n; s++) {
6     if (color[s] != -1) continue;
7

```

```

8     queue<int> q;
9     q.push(s);
10    color[s] = 0;
11
12    while (!q.empty()) {
13        int u = q.front();
14        q.pop();
15
16        for (int v : adj[u]) {
17            if (color[v] == -1) {
18                color[v] = color[u] ^ 1;
19                q.push(v);
20            }
21            else if (color[v] == color[u]) {
22                bipartite = false;
23            }
24        }
25    }
26 }

```

Uso:

```

1 vector<int> color(n, -1);
2
3 bool bipartite = true;
4
5 for (int s = 0; s < n; s++) {
6     if (color[s] != -1) continue;
7     // ...
8 }

```

2.4. Bellman-Ford — $O(NM)$

Algoritmo 4. Bellman-Ford

```

1 vector<ll> dist(n, INF);
2 dist[s] = 0;
3
4 for (int i = 0; i < n - 1; i++) {
5     bool changed = false;
6
7     for (auto [u, v, w] : edges) {
8         if (dist[u] != INF && dist[u] + w < dist[v]) {
9             dist[v] = dist[u] + w;
10            changed = true;
11        }
12    }
13
14    if (!changed) break;
15 }

```

2.5. Floyd-Warshall — $O(N^3)$

Algoritmo 5. Floyd-Warshall

```
1 vector<vector<ll>> dist(n, vector<ll>(n, INF));
2
3 for (int i = 0; i < n; i++)
4     dist[i][i] = 0;
5
6 for (auto [u, v, w] : edges)
7     dist[u][v] = min(dist[u][v], (ll)w);
8
9 for (int k = 0; k < n; k++)
10     for (int i = 0; i < n; i++)
11         for (int j = 0; j < n; j++)
12             if (dist[i][k] != INF && dist[k][j] != INF)
13                 dist[i][j] = min(dist[i][j],
14                                     dist[i][k] + dist[k][j]);
```

2.6. Topological Sort — $O(N + M)$

Algoritmo 6. Topological Sort (Kahn com BFS)

```
1 vector<int> indeg(n);
2
3 for (int u = 0; u < n; u++)
4     for (int v : adj[u])
5         indeg[v]++;
6
7 queue<int> q;
8
9 for (int i = 0; i < n; i++)
10     if (indeg[i] == 0)
11         q.push(i);
12
13 vector<int> topo;
14
15 while (!q.empty()) {
16     int u = q.front();
17     q.pop();
18
19     topo.push_back(u);
20
21     for (int v : adj[u]) {
22         if (--indeg[v] == 0)
23             q.push(v);
24     }
25 }
26
27 // se topo.size() < n, existe ciclo
```

2.7. SCC — Kosaraju — $O(V + E)$

Algoritmo 7. Strongly Connected Components (Kosaraju)

```
1 vector<vector<int>> g, gr;
2 vector<int> vis, ordem, comp;
3
4 void dfs1(int v) {
5     vis[v] = 1;
6     for (int u : g[v])
7         if (!vis[u])
8             dfs1(u);
9     ordem.push_back(v);
10 }
11
12 void dfs2(int v, int c) {
13     comp[v] = c;
14     for (int u : gr[v])
15         if (comp[u] == -1)
16             dfs2(u, c);
17 }
18
19 void kosaraju(int n) {
20     vis.assign(n, 0);
21     comp.assign(n, -1);
22
23     for (int v = 0; v < n; v++)
24         if (!vis[v])
25             dfs1(v);
26
27     reverse(ordem.begin(), ordem.end());
28
29     int c = 0;
30     for (int v : ordem)
31         if (comp[v] == -1)
32             dfs2(v, c++);
33 }
```

Uso:

```
1 // primeiro
2 g[u].push_back(v);
3 gr[v].push_back(u);
4
5 // comp[u] == comp[v] significa que u e v
6 // pertencem a mesma SCC
7 kosaraju(n);
```

2.8. LCA — $O(\log N)$

Encontrar o nó mais profundo que é ancestral comum a dois nós especificados.

Algoritmo 8. Lowest Common Ancestor (LCA) com Binary Lifting

```
1  const int LOG = 20;
2
3  vector<vector<int>> adj;
4  vector<array<int, LOG>> up;
5  vector<int> depth;
6
7  void dfs(int u, int p) {
8      up[u][0] = p;
9
10     for (int j = 1; j < LOG; j++)
11         up[u][j] = up[up[u][j-1]][j-1];
12
13     for (int v : adj[u]) {
14         if (v == p) continue;
15
16         depth[v] = depth[u] + 1;
17         dfs(v, u);
18     }
19 }
20
21 int lca(int u, int v) {
22     if (depth[u] < depth[v])
23         swap(u, v);
24
25     int diff = depth[u] - depth[v];
26
27     for (int j = 0; j < LOG; j++)
28         if (diff & (1 << j))
29             u = up[u][j];
30
31     if (u == v) return u;
32
33     for (int j = LOG - 1; j >= 0; j--) {
34         if (up[u][j] != up[v][j]) {
35             u = up[u][j];
36             v = up[v][j];
37         }
38     }
39
40     return up[u][0];
41 }
```

2.9. Fluxo de Dinic

Algoritmo 9. Algoritmo de Fluxo Máximo de Dinic

```
1  struct Dinic {
2      struct Edge {
3          int to, rev, cap;
```

```

4     };
5
6     int n;
7     vector<vector<Edge>> g;
8     vector<int> level, ptr;
9
10    Dinic(int n) : n(n), g(n), level(n), ptr(n) {}
11
12    void addEdge(int v, int u, int cap) {
13        Edge a{u, (int)g[u].size(), cap};
14        Edge b{v, (int)g[v].size(), 0};
15        g[v].push_back(a);
16        g[u].push_back(b);
17    }
18
19    bool bfs(int s, int t) {
20        fill(level.begin(), level.end(), -1);
21        queue<int> q;
22        q.push(s);
23        level[s] = 0;
24
25        while (!q.empty()) {
26            int v = q.front();
27            q.pop();
28
29            for (auto &e : g[v]) {
30                if (e.cap > 0 && level[e.to] == -1) {
31                    level[e.to] = level[v] + 1;
32                    q.push(e.to);
33                }
34            }
35        }
36        return level[t] != -1;
37    }
38
39    int dfs(int v, int t, int pushed) {
40        if (!pushed) return 0;
41        if (v == t) return pushed;
42
43        for (int &cid = ptr[v]; cid < (int)g[v].size(); cid++) {
44            Edge &e = g[v][cid];
45
46            if (e.cap > 0 && level[e.to] == level[v] + 1) {
47                int tr = dfs(e.to, t, min(pushed, e.cap));
48
49                if (tr) {
50                    e.cap -= tr;
51                    g[e.to][e.rev].cap += tr;
52                    return tr;

```



```

53         }
54     }
55 }
56     return 0;
57 }
58
59     int maxflow(int s, int t) {
60         int flow = 0;
61
62         while (bfs(s, t)) {
63             fill(ptr.begin(), ptr.end(), 0);
64
65             while (int pushed = dfs(s, t, INT_MAX))
66                 flow += pushed;
67         }
68
69         return flow;
70     }
71 };

```

Uso:

```

1 Dinic d(n);
2
3 d.addEdge(u, v, capacidade);
4
5 cout << d.maxflow(s, t) << '\n';

```

2.10. Dijkstra — $O((N + M) \log N)$

Algoritmo 10. Algoritmo de Dijkstra para caminhos mínimos

```

1 vector<ll> dijkstra(int s) {
2     vector<ll> dist(n, LINF);
3
4     priority_queue<
5         pair<ll, int>,
6         vector<pair<ll, int>>,
7         greater<pair<ll, int>>
8     > pq;
9
10    dist[s] = 0;
11    pq.push({0, s});
12
13    while (!pq.empty()) {
14        auto [d, u] = pq.top();
15        pq.pop();
16
17        if (d != dist[u])
18            continue;

```

```

19
20     for (auto [v, w] : adj[u]) {
21         if (dist[u] + w < dist[v]) {
22             dist[v] = dist[u] + w;
23             pq.push({dist[v], v});
24         }
25     }
26 }
27
28 return dist;
29
30 }

```

Uso:

```

1 vector<vector<pair<int, int>>> adj(n);
2
3 for (int i = 0; i < m; i++) {
4     int u, v, w;
5     cin >> u >> v >> w;
6
7     adj[u].push_back({v, w});
8     adj[v].push_back({u, w});
9
10
11 }
12
13 vector<ll> dist = dijkstra(0);
14
15 cout << dist[v] << '\n';

```

2.11. Representação de Grafos

Sem peso e não direcionado:

```

1 vector<vector<int>> adj(n);
2
3 for (int i = 0; i < m; i++) {
4     int u, v;
5     cin >> u >> v;
6
7     adj[u].push_back(v);
8     adj[v].push_back(u);
9
10
11 }

```

Sem peso e direcionado:

```

1 vector<vector<int>> adj(n);

```

```

2
3 for (int i = 0; i < m; i++) {
4 int u, v;
5 cin >> u >> v;
6
7 adj[u].push_back(v);
8
9
10 }

```

Com peso e não direcionado:

```

1 vector<vector<pair<int, ll>>> adj(n);
2
3 for (int i = 0; i < m; i++) {
4 int u, v;
5 ll w;
6 cin >> u >> v >> w;
7
8 adj[u].push_back({v, w});
9 adj[v].push_back({u, w});
10
11
12 }

```

Com peso e direcionado:

```

1 vector<vector<pair<int, ll>>> adj(n);
2
3 for (int i = 0; i < m; i++) {
4 int u, v;
5 ll w;
6 cin >> u >> v >> w;
7
8 adj[u].push_back({v, w});
9
10
11 }

```

3. Estruturas de Dados

3.1. DSU (Disjoint Set Union)

Algoritmo 11. Disjoint Set Union (DSU)

```

1 struct DSU {
2     vector<int> p, sz;
3
4     DSU(int n) : p(n), sz(n, 1) {
5         iota(p.begin(), p.end(), 0);

```

```

6     }
7
8     int find(int x) {
9         return p[x] == x ? x : p[x] = find(p[x]);
10    }
11
12    bool unite(int a, int b) {
13        a = find(a);
14        b = find(b);
15
16        if (a == b) return false;
17
18        if (sz[a] < sz[b]) swap(a, b);
19
20        p[b] = a;
21        sz[a] += sz[b];
22
23        return true;
24    }
25 };

```

Uso:

```

1 DSU dsu(n);
2
3 dsu.unite(a, b);
4
5 if (dsu.find(a) == dsu.find(b))
6     cout << "mesmo componente\n";

```

3.2. Fenwick Tree

Algoritmo 12. Fenwick Tree

```

1 struct Fenwick {
2     int n;
3     vector<ll> bit;
4
5     Fenwick(int n) : n(n), bit(n + 1, 0) {}
6
7     void add(int idx, ll val) {
8         for (++idx; idx <= n; idx += idx & -idx)
9             bit[idx] += val;
10    }
11
12    ll sum(int idx) {
13        ll res = 0;
14
15        for (++idx; idx > 0; idx -= idx & -idx)
16            res += bit[idx];

```

```

17
18         return res;
19     }
20
21     ll query(int l, int r) {
22         if (l > r) return 0;
23         return sum(r) - (l ? sum(l - 1) : 0);
24     }
25 };

```

Uso:

```

1 Fenwick fw(n);
2
3 fw.add(3, 10); // a[3] += 10
4 fw.add(5, 7); // a[5] += 7
5
6 cout << fw.sum(5) << '\n'; // soma [0..5]
7 cout << fw.query(2, 5) << '\n'; // soma [2..5]

```

3.3. Soma — Segment Tree

Algoritmo 13. Segment Tree para Soma

```

1 struct SegTree {
2     int n;
3     vector<ll> st;
4
5     SegTree(vector<ll>& a) {
6         n = a.size();
7         st.resize(4 * n);
8         build(a, 1, 0, n - 1);
9     }
10
11     void build(vector<ll>& a, int p, int l, int r) {
12         if (l == r) {
13             st[p] = a[l];
14             return;
15         }
16
17         int m = (l + r) / 2;
18
19         build(a, 2*p, l, m);
20         build(a, 2*p+1, m+1, r);
21
22         st[p] = st[2*p] + st[2*p+1];
23     }
24
25     void update(int p, int l, int r, int idx, ll val) {
26         if (l == r) {

```

```

27         st[p] = val;
28         return;
29     }
30
31     int m = (l + r) / 2;
32
33     if (idx <= m)
34         update(2*p, l, m, idx, val);
35     else
36         update(2*p+1, m+1, r, idx, val);
37
38     st[p] = st[2*p] + st[2*p+1];
39 }
40
41 ll query(int p, int l, int r, int ql, int qr) {
42     if (qr < l || r < ql)
43         return 0;
44
45     if (ql <= l && r <= qr)
46         return st[p];
47
48     int m = (l + r) / 2;
49
50     return query(2*p, l, m, ql, qr)
51         + query(2*p+1, m+1, r, ql, qr);
52 }
53
54 void update(int idx, ll val) {
55     update(1, 0, n-1, idx, val);
56 }
57
58 ll query(int l, int r) {
59     return query(1, 0, n-1, l, r);
60 }
61 };

```

4. Strings

4.1. KMP — $O(N + M)$

Algoritmo 14. Knuth-Morris-Pratt (KMP)

```

1 vector<int> prefix_function(string s) {
2     int n = s.size();
3     vector<int> pi(n);
4
5     for (int i = 1; i < n; i++) {
6         int j = pi[i - 1];
7
8         while (j > 0 && s[i] != s[j])

```

```

9         j = pi[j - 1];
10
11         if (s[i] == s[j])
12             j++;
13
14         pi[i] = j;
15     }
16
17     return pi;
18 }

```

5. Matemática

5.1. Fórmulas Matemáticas

5.1.1. Distância Euclidiana

Entre (x_1, y_1) e (x_2, y_2) :

Algoritmo 15. Distância Euclidiana

```

1 double dist(double x1, double y1, double x2, double y2) {
2     return hypot(x1 - x2, y1 - y2);
3 }

```

5.1.2. Distância Manhattan

Algoritmo 16. Distância Manhattan

```

1 ll manhattan(ll x1, ll y1, ll x2, ll y2) {
2     return abs(x1-x2) + abs(y1-y2);
3 }

```
